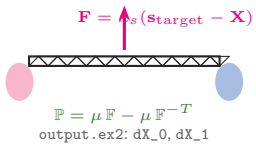
GETTING STARTED WITH IBFE

A finite element structure in IBAMR: the mesh, the stress and force functions, the input keys, the first example run three ways, and the ExodusII output in VisIt



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part IV • Chapter 17

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. say what IBFE changes about the structure: a finite element mesh with a material law in place of points joined by springs and beams, forces from a stress tensor and from body and surface forces, and spreading from quadrature points rather than nodes;
2. name the four things an IBFE program needs (a mesh, a stress function, a force function, the registration of both) and find each of them in `main.cpp` of example IBFE/01;
3. write a tether (target) force in the reference configuration, the IBFE equivalent of chapter 9a's target points, and move a plate up, down, halfway and around with it;
4. read the IBFE-specific lines of the input file and say which of them matter for a mesh read from a file;
5. build and run IBFE/01, find `output.ex2`, and draw the plate at its current position in VisIt with the `dX` expression and the Displace operator;
6. tell the 2014–2024 versions of this example apart from the current one: Makefile flags, start-up calls, signatures and a rotation that was only approximately a rotation.

> Everything in Part III was points, springs and beams. From here on the structure is a volume with a material: IBAMR's `IBFEMethod` with libMesh.

WHERE THIS CHAPTER COMES FROM

- Alex Hoover's 2014 tutorial "1-IBFE Tutorial: Getting Started" (18 slides, UNC, edited 2017): the first IBFE example, its input keys, the Makefile flags for the `ibamr/3803` and `ibamr/2016-11` modules on KillDevil, three tether examples (full plate, half plate, rotation) and a sinusoidal-force example.
- The 2018 MATH 769 version "1-IBFE Getting Starting and Visualizing a Mesh" (27 slides) and the 2020–2024 lab versions (26 slides, last edited 2024-07-30 for the Arizona HPC): the same with the VisIt part (Displace, the `dX` expression) merged in and the Makefile reduced to one slide.
- The READMEs shipped with the examples (Hoover 2014; Julian Wag 2016): the only description of how the `.mat` mesh was made in MATLAB's PDE Toolbox, and a `.poly` generator that describes a different rectangle (slide 46).
- New here: the `IBTKInit` start-up, the current nine-argument callback signature, surface forces and pressures (listed in every version, shown in none), what the input keys do to a mesh read from a file, the exact rotation, and the measured results of the three runs.

> The example is shipped as `examples/IBFE/01-tethered-plate`: one program with an input key `MOTION` in place of the course's three folders. It runs in under a minute.

WHAT IBFE IS, IN ONE SLIDE

- The structure is a finite element mesh: triangles or quadrilaterals in 2D, tetrahedra or hexahedra in 3D, with nodes and a reference (initial) configuration $\mathbf{s}$. Its current position is $\mathbf{X}(\mathbf{s}, t)$ and its deformation gradient is $\mathbb{F} = \partial \mathbf{X} / \partial \mathbf{s}$.
- The structure's elastic force comes from a material law: a first Piola–Kirchhoff stress $\mathbb{P}(\mathbb{F})$ that you write as a C++ function. Springs and beams are gone.
- Body forces (per unit reference volume, like chapter 9a's target points) and surface forces and pressures (per unit reference area, on the mesh's boundary) are further C++ functions.
- Forces are spread to the fluid grid from quadrature points of the elements, not from the nodes; the number of points is set by the input so that no fluid leaks through the structure.
- The fluid side is unchanged: the same `INSStaggeredHierarchyIntegrator`, the same `IBExplicitHierarchyIntegrator`, the same input blocks as chapters 5–7.

- Reference: B. E. Griffith and X. Luo, *Hybrid finite difference/finite element immersed boundary method*, Int. J. Numer. Meth. Biomed. Eng. 33 (2017) e2888; preprint arXiv:1612.05916. The solid mechanics is in the theory repository's IBFE chapters.
- In IBAMR: `IBFEMethod` (the IB operations for a finite element structure) built on `FEMechanicsBase` (the mechanics: stress, force and mapping functions) and `IBTK::FEDataManager` (the data). The mesh library is libMesh (1.7.8 in autoibamr 0.19.0).

> Think of chapter 9a's target points: a force toward a prescribed position. In IBFE the prescribed position is a function of the reference position $\mathbf{s}$, and the force acts on every point of the volume.

WHAT YOU WILL NEED

1. A mesh. Either built inside the program with libMesh's generators (`MeshTools::Generation::build_square`, `build_cube`; IBAMR's own examples do this) or made outside and read from a file: MATLAB's PDE Toolbox (this chapter), gmsh (chapter 18, examples IBFE/05 and IBFE/10), TetGen and Triangle, Coreform Cubit (formerly Trelis and Bolt), Salome, or anything that writes a format libMesh reads (slide 33).
2. A material model: a function that returns the first Piola–Kirchhoff stress $\mathbb{P}$ from the deformation gradient $\mathbb{F}$ (here a neo-Hookean solid in two parts, slide 27; chapter 19 has the theory and other laws).
3. Forces: a body force function (the tether of this chapter; chapter 20 adds active fibre tension), and if you need them a surface pressure or a surface force function (slide 30).
4. The registration of these with `IBFEMethod`, and the two input blocks that configure it (slide 11).

> The 2014 slide also sent you to a solid-mechanics review ("look for *Stress Measures for Large Deformations*"). That review is the theory repository's IBFE chapter now; the link in the old slides, to a 2015 lecture-notes PDF at Scuola Superiore Sant'Anna, still works but is third-party material.

IMPORTANT LINKS, UPDATED

what	where (2026)
the IBFE paper	Griffith & Luo 2017, https://onlinelibrary.wiley.com/doi/full/10.1002/cnm.2888; preprint https://arxiv.org/abs/1612.05916. The 2014 slide linked a PDF at `math.nyu.edu/phd_students/griffith`, which is gone.
libMesh documentation	https://libmesh.github.io/doxygen/ (the 2014–2020 slides pointed at `libmesh.sourceforge.net`, which no longer serves it).
reading meshes	the `MeshInput` class and the format list in `NameBasedIO` (slide 33); `libMesh::Mesh::read()`.
tensors	`libMesh::TensorValue` and `VectorValue`: the types of $\mathbb{F}$, $\mathbb{P}$ and $\mathbf{F}$ in every callback.
IBAMR's libMesh helpers	`ibtk/include/ibtk/libmesh_utilities.h` on GitHub, https://github.com/IBAMR/IBAMR; it holds the callback signatures of slide 17 and `tensor_inverse_transpose`. The old link went to Google Code, which closed in 2016.
IBAMR's IBFE examples	`examples/IBFE/explicit/ex0` to `ex11` in the source tree (chapter 25 tours them).
the example of this chapter	`examples/IBFE/01-tethered-plate` in the tutorial's repository.

Links checked October 2026.

EXAMPLE IBFE/01: A TETHERED PLATE, THREE WAYS

A thin elastic plate, 1.0 long and 0.0312 thick, sits at the centre of a 2×2 periodic box of fluid (density 1, viscosity 0.01). A tether force pulls every point of the plate toward a target position that moves. The input key `MOTION` chooses the motion:

`MOTION`	what the tether does
`"FULL"`	the whole plate moves up at speed 0.5 until $t = 1$, down at 0.5 until $t = 3$, then is released.
`"HALF"`	the same motion, but only the right half (reference $x > 0$) is tethered; the left half is passive and the plate bends.
`"ROTATE"`	a quarter turn counter-clockwise about the centre during $0 < t < 1$, then released.

In the course these were three folders with nearly identical programs: `TetherForceFullPlate2D`, `TetherForceHalfPlate2D` and `TetherForceFullPlate2D-Rotate` (the 2014 deck also had `SinusoidalForce2D`, slide 25).

EXAMPLE IBFE/01 (2): THE FILES

file	what it is
`main.cpp`	the program: the tether force and the two stress functions in `namespace ModelData`, then `main()`
`input2d`	physical, grid, solver and output settings, and `MOTION`, `SHEAR_MODULUS`, `KAPPA_S`, `MESH_FILENAME`
`IBFE_Mesh2D_128.mat`	the plate's mesh, 101 nodes and 130 triangles, in libMesh's MATLAB format
`CMakeLists.txt`, `job.slurm`, `README.md`	the build, the Puma job, the measured results
`extras/`	the course's `Beam_IBFE_Mesh2D.m` (and a Python port) and the `.poly` file it writes; not used (slide 46)

THE MESH: 101 NODES, 130 TRIANGLES, ONE FILE



- `TRI3` elements: triangles with three nodes, linear shape functions. Three rows of nodes across the thickness ($y = \pm 0.0156$ and 0), 33 nodes along each edge, so the triangles' edges are about 0.03 long: the fine grid spacing $\mathrm{DX} = 2/128 = 0.0156$ times two.
- The 2014 slide's rule of thumb: "Old: mesh edge length $ds = 2\,dx$ (as opposed to $\frac{1}{2}dx$); new: it seems like $ds = dx$ works better." Chapter 18 reconciles this with the `MFAC` rule of the examples that build their own mesh; for a read-in mesh the spacing is whatever the file has.
- The mesh is made outside IBAMR, in MATLAB's PDE Toolbox, and written in libMesh's MATLAB format (`.mat`, a plain text file; not a MATLAB binary `.mat`): slide 10.

VisIt: the Mesh plot of `output.ex2` at $t = 0$, zoomed to the plate.

THE `.MAT` FILE, AND WHAT LIBMESH DOES WITH IT

The file is numbers separated by spaces: the number of nodes and of elements; then x y for each node; then, for each triangle, its three node numbers (counted from 1) and the subdomain number the PDE Toolbox gives it (all 1 here). The first and last values of `IBFE_Mesh2D_128.mat`:

```
101 130 0.500000 0.015600 0.500000 -0.015600 -0.500000 -0.015600 -0.500000 0.015600 0.500000
  0.000000 0.469697 -0.015600 ...
... 41 74 101 1 34 100 101 1
```

- libMesh's `MatlabIO::read_stream` reads exactly this: "PDE toolkit only works in 2D", so the mesh dimension is set to 2, then `nNodes`, `nElem`, the coordinates, the connectivity. There is no element-type field: the format is triangles only.
- The reader is chosen by the file's extension (slide 33); `.mat` means "Matlab triangular ASCII file" in libMesh's list. Nothing in IBAMR looks inside.
- How it was made (the 2016 README's recipe, the only one in the archive): slide 45.

> The four corner nodes come first, then the edges, then the interior: that is the PDE Toolbox's order, and the tutorial's VisIt figures do not depend on it. The order of the four corners you give `pdepoly` does matter: the polygon must not cross itself.

INPUT FILE: THE IBFE-SPECIFIC LINES

```
MESH_FILENAME = "IBFE_Mesh2D_128.mat"     // libMesh MATLAB-format mesh of the plate
SHEAR_MODULUS = 10.0                      // shear modulus mu of the neo-Hookean plate
KAPPA_S       = 1.0e6                     // stiffness of the tether force
// MFAC and ELEM_TYPE are used by examples that build their mesh in the program
// (for example IBAMR's IBFE/explicit/ex2). They have no effect on a mesh read
// from a file, like this one. Its element size was chosen when it was made.
MFAC = 2.0                                // ratio of Lagrangian mesh width to Cartesian mesh width
ELEM_TYPE = "TRI3"                        // type of element to use for structure discretization
PK1_DEV_QUAD_ORDER = "FIFTH"              // quadrature order for the deviatoric stress
PK1_DIL_QUAD_ORDER = "THIRD"              // quadrature order for the dilational stress
IB_DELTA_FUNCTION          = "IB_4"       // the type of smoothed delta function to use for Lagrangian-Eulerian interaction
SPLIT_FORCES               = TRUE         // whether to split interior and boundary forces
USE_JUMP_CONDITIONS        = FALSE        // whether to impose pressure jumps at fluid-structure interfaces
USE_CONSISTENT_MASS_MATRIX = TRUE         // whether to use a consistent or lumped mass matrix
IB_POINT_DENSITY           = 2.0          // approximate density of IB quadrature points for Lagrangian-Eulerian interaction

IBFEMethod {
   IB_delta_fcn               = IB_DELTA_FUNCTION
   split_forces               = SPLIT_FORCES
   use_jump_conditions        = USE_JUMP_CONDITIONS
   use_consistent_mass_matrix = USE_CONSISTENT_MASS_MATRIX
   IB_point_density           = IB_POINT_DENSITY
}
Main {
   viz_writer                  = "VisIt","ExodusII"   // VisIt files for the fluid, ExodusII for the plate
   viz_dump_interval           = 100
   viz_dump_dirname            = "viz_IB2d"
   visit_number_procs_per_file = 1
}
```

INPUT FILE (2): WHAT EACH KEY DOES

key	meaning (from `IBFEMethod::getFromInput` in 0.19.0)
`IB_point_density`	the approximate number of quadrature points per fine grid cell used to interpolate velocity to and spread force from the elements (the same value for both unless `interp_point_density` and `spread_point_density` are given). With `use_adaptive_quadrature` the rule's order is raised element by element to reach it. More points: no leaks, more cost.
`IB_delta_fcn`	the kernel, as in Part III (`IB_4` here); `interp_delta_fcn` and `spread_delta_fcn` can differ.
`split_forces`	spread the interior (volumetric) force and the boundary (jump) force separately; sets both `split_normal_force` and `split_tangential_force`.
`use_jump_conditions`	impose the pressure jump across the boundary (needs `split_forces`).
`use_consistent_mass_matrix`	project the interpolated velocity with the consistent mass matrix rather than a lumped one.

INPUT FILE (3): THE KEYS THE PROGRAM READS, AND WHAT THE OLD COMMENTS GOT WRONG

key	meaning
`PK1_DEV_QUAD_ORDER`, `PK1_DIL_QUAD_ORDER`	read by *the program*, not by `IBFEMethod`, and passed on as each stress function's `quad_order` (slide 28).
`MFAC`, `ELEM_TYPE`	read by programs that build their own mesh (`ds = MFAC*dx`; the element type); no effect on a mesh read from a file. The 2014–2024 comments "It seems like 1.0 is better" next to `MFAC = 2.0` and `IB_POINT_DENSITY = 2.0` were never acted on; the shipped values are what the runs of slides 41–43 used.
`viz_writer = "VisIt","ExodusII"`	the second word makes `AppInitializer` supply an ExodusII file name (`viz_IB2d/output.ex2`); without it `uses_exodus` is false and no plate file is written.

THE PROGRAM (1): THE START-UP, THEN AND NOW

Current (`IBTKInit`, IBAMR 0.10 and later; the example's `main.cpp`):

```
int main(int argc, char* argv[])
{
    // Initialize MPI, PETSc, libMesh, SAMRAI and IBAMR. IBTKInit also shuts
    // them down when it goes out of scope at the end of main().
    IBTKInit ibtk_init(argc, argv, MPI_COMM_WORLD);
    const LibMeshInit& init = ibtk_init.getLibMeshInit();
    { // cleanup dynamically allocated objects prior to shutdown
        Pointer<AppInitializer> app_initializer = new AppInitializer(argc, argv, "IB.log");
        Pointer<Database> input_db = app_initializer->getInputDatabase();
```

The course versions (2014–2020) did the same by hand, and the 2020 "main file" lecture (chapter 23) explains each line:

```
    LibMeshInit init(argc, argv);
    SAMRAI_MPI::setCommunicator(PETSC_COMM_WORLD);
    SAMRAI_MPI::setCallAbortInSerialInsteadOfExit();
    SAMRAIManager::startup();
    ...
    SAMRAIManager::shutdown();
    PetscFinalize();
```

- `IBTKInit(argc, argv, communicator, petsc_file, petsc_database)`: the constructor starts MPI, PETSc, SAMRAI and (when IBAMR was built with libMesh) libMesh; the destructor stops them. A second `IBTKInit` in the same program is a run-time error.
- The old `#include <IBAMR_config.h>` and `<IBTK_config.h>` are gone; `<SAMRAI_config.h>` stays.

THE PROGRAM (2): NAMESPACE MODELDATA

The material and the forces live in a namespace above `main()`, as in every IBAMR example. The parameters are static variables that `main()` overwrites from the input file:

```
namespace ModelData
{
static const double PI = 3.14159265358979323846;
// Shear modulus of the neo-Hookean plate material, and the stiffness of the
// tether that pulls each point of the plate toward its target position.
// Both can be changed in input2d (SHEAR_MODULUS and KAPPA_S).
static double mu = 10.0;
static double kappa_s = 1.0e6;
// How the target positions move (input key MOTION).
enum class Motion { FULL, HALF, ROTATE };
static Motion motion = Motion::FULL;
...
} // namespace ModelData
using namespace ModelData;
```

- A namespace, not a class: the callbacks are plain functions (slide 17), and IBAMR calls them without any object of yours, so the parameters they need must be visible to them: static variables in the namespace, or a `ctx` pointer.

THE PROGRAM (3): `MAIN()` READS THE MODEL PARAMETERS

In `main()`, after the input database exists:

```
const string motion_name = input_db->getStringWithDefault("MOTION", "FULL");
if (motion_name == "FULL") ModelData::motion = Motion::FULL;
else if (motion_name == "HALF") ModelData::motion = Motion::HALF;
else if (motion_name == "ROTATE") ModelData::motion = Motion::ROTATE;
else TBOX_ERROR("Unknown MOTION = \"" << motion_name << "\" in the input file.\n" << "Valid options are: FULL, HALF, ROTATE");
ModelData::mu = input_db->getDoubleWithDefault("SHEAR_MODULUS", ModelData::mu);
ModelData::kappa_s = input_db->getDoubleWithDefault("KAPPA_S", ModelData::kappa_s);
```

- The 2014 version had `static const double mu = 10.0; //Elastic modulus of the beam` and `kappa_s = 1.0e6`, fixed in the code. `mu` is the shear modulus; `ModelData::` is written out in `main()` because the fluid's `mu` is also in scope there.

THE CALLBACK SIGNATURE: NINE ARGUMENTS, TWO NAMINGS

Every force or stress function has the shape of `IBTK::VectorMeshFcnPtr` (or `TensorMeshFcnPtr` for a stress), from `libmesh_utilities.h`:

```
using VectorMeshFcnPtr = void (*)(libMesh::VectorValue<double>& F,          // output: the force (or stress PP)
        const libMesh::TensorValue<double>& FF,                             // deformation gradient dX/ds
        const libMesh::Point& x,                                            // current position
        const libMesh::Point& X,                                            // reference position
        libMesh::Elem* elem,                                                // the element
        const std::vector<const std::vector<double>*>& system_var_data,     // values of registered FE systems
        const std::vector<const std::vector<libMesh::VectorValue<double>>*>& system_grad_var_data,
        double data_time,                                                   // the time
        void* ctx);                                                         // your context pointer
```

IBAMR's name	the course's name	meaning
`x`	`X`	where the point is now (current, physical)
`X`	`s`	where it started (reference, Lagrangian); `s(0)` is x, `s(1)` is y
`FF`	`FF`	$\mathbb{F} = \partial \mathbf{X}/\partial \mathbf{s}$, the deformation gradient
`F`	`F`	the output: force per unit reference volume (body force), or $\mathbb{P}$ for a stress function

The course kept Hoover's names (`X` current, `s` reference) and so does the example; IBAMR's headers use `x` and `X`. Both are only names. The 2014 slide's seven-argument form with `const vector<NumericVector<double>*>& system_data` is the pre-2016 signature (slide 48).

THE TETHER FORCE: A TARGET POSITION WRITTEN IN THE REFERENCE CONFIGURATION

Chapter 9a's target point: a force $\kappa(\mathbf{X}_{\text{target}} - \mathbf{X})$ toward a prescribed position. Here the prescribed position is a function of where the point started, $\mathbf{s}$, so one formula moves the whole plate. The 2014 slides:

"$(\mathbf{s}(0), \mathbf{s}(1))$ are the (x, y) coordinates of the initial mesh. $(\mathbf{s}_{\text{dump}}(0), \mathbf{s}_{\text{dump}}(1))$ are the (x, y) coordinates of where you currently want the target points to be."

Translate up at $V = 0.5$:

```
s_dump(1) = s(1) + 0.5*time;  // only the y-coordinate changes
s_dump(0) = s(0);
```

Translate left at $V = 0.3$:

```
s_dump(1) = s(1);
s_dump(0) = s(0) - 0.3*time;  // only the x-coordinate changes
```

Then `F = kappa_s*(s_dump - X)`. Setting `s_dump = X` gives zero force: that is how the plate is released.

- `s_dump` is the course's name for the target position; the example calls it `s_target`.
- A slip that survived from 2014 to 2024: the comment "`s(1)` is the x-coordinate of the reference configuration" on the `s_dump(1)` line. `s(1)` is y; `s(0)` is x. (The 2020 deck corrected it on one slide and kept it on two.)
- $\kappa_s = 10^6$ is stiff: the plate follows the target to better than 0.01 (slide 43). $\kappa_s = 10^4$ lags and overshoots.
- The force is a *body* force, per unit reference volume, registered with `registerLagBodyForceFunction` (slide 28). It acts on every quadrature point of every element, not only on the nodes.

> A moving target in the reference frame is the IBFE way to prescribe motion. Chapter 21's two-coral example and chapter 22's flow tank use the same trick in 3D.

THE TETHER FORCE (2): THE EXAMPLE'S ONE FUNCTION FOR ALL THREE MOTIONS

```
// Target ("tether") force on the plate, per unit reference area.
//   s is the reference (initial) position of a point in the plate:
//     s(0) is its x-coordinate and s(1) is its y-coordinate.
//   X is where that point is now.
void target_force_function(VectorValue<double>& F, const TensorValue<double>& /*FF*/, const libMesh::Point& X,
                           const libMesh::Point& s, Elem* const /*elem*/,
                           const std::vector<const std::vector<double>*>& /*var_data*/,
                           const std::vector<const std::vector<VectorValue<double> >*>& /*grad_var_data*/,
                           double time, void* /*ctx*/)
{
    libMesh::Point s_target = X;            // the default: no force
    switch (motion)
    {
    case Motion::FULL:
    case Motion::HALF:
        // With HALF, only points that start in the right half (s(0) > 0) are
        // tethered; the left half feels no target force at any time.
        if (motion == Motion::HALF && s(0) <= 0.0) break;
        if (time < 1.0)      { s_target(0) = s(0); s_target(1) = s(1) + 0.5 * time; }               // up at 0.5
        else if (time < 3.0) { s_target(0) = s(0); s_target(1) = s(1) + 0.5 + 0.5 * (1.0 - time); } // down at 0.5
        // for t > 3 the target is X itself: the plate moves only with the fluid
        break;
    // ... continued on the next slide
```

THE TETHER FORCE (3): THE ROTATE CASE AND THE FORCE

```
    case Motion::ROTATE:
        if (time < 1.0)
        {   // turn counterclockwise about the origin by a quarter turn over 0 < t < 1
            const double theta = 0.5 * PI * time;
            s_target(0) = s(0) * std::cos(theta) - s(1) * std::sin(theta);
            s_target(1) = s(0) * std::sin(theta) + s(1) * std::cos(theta);
        }
        break;                              // for t > 1 the plate is released
    }
    F = kappa_s * (s_target - X);
}
```

- The `switch` replaces the course's three programs: one function, one input key. `s_target = X` before the `switch` is the "no force" default that every released or untethered point falls through to.
- `F` is the force per unit reference area (2D) or volume (3D) at the point; IBAMR multiplies by the element's quadrature weights and spreads it.

FULL AND HALF: THE 2014 CODE, SIDE BY SIDE WITH THE EXAMPLE'S

Hoover's `TetherForceFullPlate2D` (2014, slide 12):

```
libMesh::Point s_dump;   //s_dump is the target point position
if(time<1)  //move the beam up with a speed of 0.5 from 0<t<1
{
  s_dump(1)=s(1)+.5*time;
  s_dump(0)=s(0);
}
else if(time<3)  //move the beam down with a speed of -0.5 for 1<
  t<3
{
  s_dump(1)=s(1)+.5+.5*(1-time);
  s_dump(0)=s(0);
}
else  //set the target point location to the actual location
{     //of the boundary (X) to make force equal to zero.
  s_dump(1)=X(1);
  s_dump(0)=X(0);
}
F = kappa_s*(s_dump-X);
return;
```

`TetherForceHalfPlate2D` (2014, slide 14): "This if statement only applies a force if you are on the right side."

```
if(s(0)>0){
  libMesh::Point s_dump;
  if(time<1) { s_dump(1)=s(1)+.5*time; s_dump(0)=s(0); }
  else if(time<3){ s_dump(1)=s(1)+.5+.5*(1-time); s_dump(0)=s
  (0); }
  else{ s_dump(1)=X(1); s_dump(0)=X(0); }
  F = kappa_s*(s_dump-X);
}
else{
  F.zero();   //Else the force is set to zero.
}
return;
```

FULL AND HALF (2): WHAT THE TWO VERSIONS SHARE

- `F.zero()` and `F = kappa_s*(X - X)` are the same force. The example's `s_target = X` default and the early `break` fold the three versions into one `switch`.
- "The movement of the other half of the plate is purely passive (i.e. PK1 stress)": the left half is carried by the material law alone. That is why the plate bends (slide 42).
- The 2014 deck placed the centre of the plate at the origin, so `s(0) > 0` is the right half; the mesh file confirms it ($x \in [-0.5, 0.5]$).
- The 2014 text for the half plate: "Again using a body force acting as tether force defined in the ModelData namespace, I now have half the plate move up to $.5$ (in 1 s) then down 1.0 (in 2 s), after which I turn off the force (for 1 s)."

ROTATE: A ROTATION THAT WAS ONLY A ROTATION ON THE CENTRE LINE

Hoover's `TetherForceFullPlate2D-Rotate` (2014, slide 17; unchanged through 2024):

```
if(time<1)
{
  s_dump(1)=s(1)+s(0)*sin(2*3.14*0.5*time);  //y-coordinate gets
  moved
                //more the farther along the beam you are
  //should really define PI here
  s_dump(0)=s(0)*cos(2*3.14*0.5*time); //adjustment for x-
  coordinate
}
else //turns off all forcing, target coordinates set to current
{
  s_dump(1)=X(1);
  s_dump(0)=X(0);
}
F = kappa_s*(s_dump-X);
```

With $\theta = 2 \cdot 3.14 \cdot 0.5\, t \approx \pi t$: a half turn in one second, and

$$x' = x\cos\theta, \qquad y' = y + x\sin\theta.$$

The example's version (slide 20), with $\theta = \pi t/2$:

```
const double theta = 0.5 * PI * time;
s_target(0) = s(0) * std::cos(theta) - s(1) * std::sin(theta);
s_target(1) = s(0) * std::sin(theta) + s(1) * std::cos(theta);
```

$$x' = x\cos\theta - y\sin\theta, \qquad y' = x\sin\theta + y\cos\theta.$$

The 2014 and 2020 text: "The goal here is to spin the beam about its center for $t < 1$, and then release it."

ROTATE (2): WHAT THE OLD FORMULA DID, AND WHAT THE EXAMPLE DOES

- A rotation by θ is $x' = x\cos\theta - y\sin\theta$, $y' = x\sin\theta + y\cos\theta$. The 2014 formula drops $-y\sin\theta$ and replaces $y\cos\theta$ by y: exact for $y = 0$ (the centre line), and off by up to the plate's half-thickness, 0.0156, elsewhere. With $\kappa_s = 10^6$ the plate was pulled into a slightly sheared shape, not a rotated one.
- $2 \cdot 3.14$ for 2π ("should really define PI here") is a 0.05 per cent error in the angle; the example defines `PI` to full precision.
- The 2014 text said "spin the beam about its center for $t < 1$", the formula gives a half turn ($\theta(1) = \pi$); the 2020 and 2024 decks said the same. The example makes it a quarter turn ($\theta = \pi t/2$) so that the released plate's drift is visible against the start (slide 42); set `theta = PI * time` for the half turn.
- The target of a point at the origin never moves, so the centre of the plate stays put: the measurement on slide 43 confirms it to three digits.

THE FOURTH 2014 EXAMPLE: A SINUSOIDAL BODY FORCE ON THE PLATE'S ENDS

The 2014 deck's slide 15, "`Example_IBFE_SinusoidalForce2D`": "Here I define a body force that is sinusoidally forcing our plate. The forced elements lie between -0.25 to 0.25 on the horizontal axis of the reference configuration." The code was never on a slide, and the 2018 and later decks dropped it; the folder survives as `Example_IBFE_SinusoidalForce2D_2016-11.zip` in the course's 2016-11 example set, and its `main.C` (2017-02-09) forces the *outer* quarters, `abs(s(0)) > .25`, not the middle half the slide describes:

```
static const double mu = 10.0;
static double kappa_s = 3.0e1;          // the force amplitude, per unit reference area
static const double freq = 0.5;         // half a cycle per unit time
static const double PI = acos(-1);
void sinusoidal_force_function(VectorValue<double>& F, const TensorValue<double>& /*FF*/, const libMesh::Point& X,
                               const libMesh::Point& s, Elem* const /*elem*/,
                               const vector<NumericVector<double>*>& /*system_data*/, double time, void* /*ctx*/)   // the 2016
  signature
{
  if(abs(s(0)) >.25){
  libMesh::Point s_dump;
  s_dump(0)=0;
  s_dump(1)=1;                          // a unit vector up
  F = kappa_s*sin(2*PI*time*freq)*s_dump;
} else{
  F.zero();
 }
    return;
}
```

THE FOURTH 2014 EXAMPLE (2): WHAT THE CODE DOES, AND HOW TO RUN IT TODAY

- A vertical force $30\sin(\pi t)$ per unit reference area on the two ends, nothing on the middle: the ends flap up and down and the free middle follows through the material law. The variable names are the tether's (`kappa_s`, `s_dump`) reused for an amplitude and a direction; the slide's "between -0.25 and 0.25" is the complement of what the code does.
- The plate has no tether here, so this is the one course example where the plate is free and driven by a body force alone; chapter 19 returns to it. To run it today: the nine-argument signature, `std::abs`, and `LagBodyForceFcnData` as on slide 28; only one body force function can be registered per part (the header: "It is *NOT* possible to register multiple body force functions with this class"), so a plate that is both tethered and shaken needs both terms in one function. Exercise 2 (slide 50) is to run it.

THE STRESS FUNCTIONS: A NEO-HOOKEAN PLATE IN TWO PARTS

```
// Stress tensor functions. The first Piola-Kirchhoff stress of the neo-Hookean
// material is split into two parts. The deviatoric part is mu*FF. The
// dilational part, -mu*FF^{-T}, makes the undeformed plate stress-free: at
// FF = I the two parts cancel.
void PK1_dev_stress_function(TensorValue<double>& PP, const TensorValue<double>& FF, const libMesh::Point& /*X*/,
                             const libMesh::Point& /*s*/, Elem* const /*elem*/,
                             const std::vector<const std::vector<double>*>& /*var_data*/,
                             const std::vector<const std::vector<VectorValue<double> >*>& /*grad_var_data*/,
                             double /*time*/, void* /*ctx*/)
{
    PP = mu * FF;
}
void PK1_dil_stress_function(TensorValue<double>& PP, const TensorValue<double>& FF, ...same arguments...)
{
    PP = -mu * tensor_inverse_transpose(FF, NDIM);
}
```

- $\mathbb{P} = \mu\,\mathbb{F} - \mu\,\mathbb{F}^{-T}$: the 2014 slide's "Neo-Hookean model: think springs". At $\mathbb{F} = \mathbb{I}$ the two parts cancel, so the plate as meshed carries no stress; stretch it and $\mu\mathbb{F}$ pulls back.
- Why two functions: IBAMR integrates each registered stress with its own quadrature rule. The smooth deviatoric part gets `PK1_DEV_QUAD_ORDER = "FIFTH"`, the inverse-transpose part `"THIRD"`; chapter 19 explains the choice and the other material laws (and corrects the 2014 claim that $-\mu\mathbb{F}^{-T}$ "enforces incompressibility": it does not, it only makes the reference state stress-free).
- `tensor_inverse_transpose(FF, NDIM)` is IBAMR's helper from `libmesh_utilities.h`; `mu` is `ModelData::mu`, the shear modulus.

REGISTERING THE FUNCTIONS

```
// Configure the IBFE solver: register the stress and force functions,
// then set up the finite element systems.
IBFEMethod::LagBodyForceFcnData target_force_data(target_force_function);
IBFEMethod::PK1StressFcnData PK1_dev_stress_data(PK1_dev_stress_function);
IBFEMethod::PK1StressFcnData PK1_dil_stress_data(PK1_dil_stress_function);
PK1_dev_stress_data.quad_order = Utility::string_to_enum<libMesh::Order>(input_db->getStringWithDefault("PK1_DEV_QUAD_ORDER", "THIRD
  "));
PK1_dil_stress_data.quad_order = Utility::string_to_enum<libMesh::Order>(input_db->getStringWithDefault("PK1_DIL_QUAD_ORDER", "FIRST
  "));
ib_method_ops->registerPK1StressFunction(PK1_dev_stress_data);
ib_method_ops->registerPK1StressFunction(PK1_dil_stress_data);
ib_method_ops->registerLagBodyForceFunction(target_force_data);
ib_method_ops->initializeFEEquationSystems();
EquationSystems* equation_systems = ib_method_ops->getFEDataManager()->getEquationSystems();
```

- A `*FcnData` struct wraps each function pointer with an optional `ctx` pointer and the list of FE systems whose values the function wants (`system_data`; chapter 20 uses it for the fibre direction). The stress structs also carry the quadrature rule.
- `initializeFEEquationSystems()` must come after the registrations and before `initializeFEData()`.

REGISTERING THE FUNCTIONS (2): THE DATA STRUCTS, AND HOW MANY OF EACH

struct (in `FEMechanicsBase`)	fields, and how many per part
`PK1StressFcnData`	`fcn`, `system_data`, `ctx`, `quad_type`, `quad_order`; any number (each call `push_back`s; a missing `quad_order` takes the part's default)
`LagBodyForceFcnData`	`fcn`, `system_data`, `ctx`; one (a second call replaces the first)
`LagSurfacePressureFcnData`, `LagSurfaceForceFcnData`	`fcn`, `system_data`, `ctx`; one each (slide 30)
`CoordinateMappingFcnData`	`fcn`, `ctx`: initial physical position from the reference position (chapter 22's translated mesh)
`InitialVelocityFcnData`	`fcn`, `ctx`: a starting velocity for the solid (zero if none)

The second argument of every `register*` call is the part number (default 0); with several meshes (chapter 21) each part has its own functions. `libMesh::Order` replaced the course's `libMeshEnums::Order` in 2016.

SURFACE FORCES AND PRESSURES: LISTED IN EVERY VERSION, SHOWN IN NONE

The 2014 slide 3 listed “Body Force, Surface Force”; no course deck ever showed one. They act on the *boundary* of the mesh (per unit reference area) and have twelve arguments: the nine of slide 17 plus the current and reference normals and the side of the element:

```
using ScalarSurfaceFcnPtr = void (*)(
        double& F,                                  // output: a pressure p (the force is -p n)
        const libMesh::VectorValue<double>& n,      // current unit normal
        const libMesh::VectorValue<double>& N,      // reference unit normal
        const libMesh::TensorValue<double>& FF,     // deformation gradient
        const libMesh::Point& x,                    // current position
        const libMesh::Point& X,                    // reference position
        libMesh::Elem* elem,                        // the element
        unsigned short int side,                    // which of its sides
        const std::vector<const std::vector<double>*>& system_var_data,
        const std::vector<const std::vector<libMesh::VectorValue<double>>*>& system_grad_var_data,
        double data_time,
        void* ctx);
// VectorSurfaceFcnPtr: the same, with libMesh::VectorValue<double>& F (a force vector) as the output
```

SURFACE FORCES AND PRESSURES (2): A PRESSURE LOAD, AND WHEN TO USE WHICH

A pressure load p_0 on every boundary side of the plate, and its registration:

```
void surface_pressure_function(double& P,
                               const VectorValue<double>& /*n*/, const VectorValue<double>& /*N*/,
                               const TensorValue<double>& /*FF*/,
                               const libMesh::Point& /*x*/, const libMesh::Point& /*X*/,
                               Elem* const /*elem*/, const unsigned short int /*side*/,
                               const std::vector<const std::vector<double>*>& /*var_data*/,
                               const std::vector<const std::vector<VectorValue<double> >*>& /*grad_var_data*/,
                               double /*time*/, void* /*ctx*/)
{
    P = 1.0;
}
IBFEMethod::LagSurfacePressureFcnData surface_pressure_data(surface_pressure_function);
ib_method_ops->registerLagSurfacePressureFunction(surface_pressure_data);
```

- Use a surface pressure for a load that acts on a face (an inflating tube, a transmural pressure); a surface force for a traction with a direction of its own. Restrict either with `side` or with the reference position `X`. Chapter 23 shows the upstream examples that use them.

READING YOUR MESH

```
// Read the finite element mesh of the plate. The .mat format is
// libMesh's MATLAB mesh format (see README.md for how it was made).
Mesh mesh(init.comm(), NDIM);
mesh.read(input_db->getStringWithDefault("MESH_FILENAME", "IBFE_Mesh2D_128.mat"));
mesh.prepare_for_use();
```

- `Mesh mesh(init.comm(), NDIM)`: a libMesh mesh on the communicator `IBTKInit` made, of dimension `NDIM` (2 for `main2d`). The 2014 slide shows `Mesh mesh(NDIM)` "for `ibamr/3803`" and the communicator form "for `ibamr/2016-11`"; only the second compiles today.
- `mesh.read(name)` chooses the reader from the extension (next slide) and fills the mesh; `prepare_for_use()` builds the neighbour and boundary information every later step needs. Call it once, after reading.
- The 2014 slide also read `DX`, `ds = MFAC*dx` and `ELEM_TYPE` here; for a mesh from a file they are unused, and the example does not read them (the input keeps them, commented, for the programs that do).
- The mesh is then handed to the method: `new IBFEMethod("IBFEMethod", db, &mesh, max_levels, register_for_restart, restart_read_dirname, restart_restore_num)`, and `ib_method_ops->initializeFEData()` after the registrations.

> "Look at the README files in the examples" (2014). The READMEs are in `examples/IBFE/*/README.md` and say, for each example, where its mesh came from.

MESH FORMATS LIBMESH READS (BY FILE EXTENSION)

extension	format (libMesh 1.7.8, `NameBasedIO`)	used in this tutorial by
`.mat`	MATLAB triangular ASCII (PDE Toolbox)	IBFE/01, 06, 12
`.msh`	gmsh (ASCII versions 2.2 and 4; never binary)	IBFE/05 triangle, IBFE/10 3D plate; chapter 18
`.e`, `.exo`, `.exd`	ExodusII (also what IBAMR writes)	IBFE/08 beam and circle; Coreform Cubit exports
`.node` + `.ele`	TetGen (3D) and Triangle (2D)	chapter 18
`.inp`	Abaqus	–
`.ucd`, `.unv`, `.gmv`, `.off`, `.mesh`	AVS UCD, I-deas Universal, General Mesh Viewer, OOGL OFF, Medit	–
`.vtu`	VTK unstructured (Paraview)	–
`.xda`, `.xdr`	libMesh's own ASCII and binary formats	`output_data()` writes `fe_mesh.*.xda`
`.cpa`, `.cpr`	libMesh checkpoint, ASCII and binary	–

- The list is printed by libMesh itself when it does not recognise an extension (`NameBasedIO::read`: "ERROR: Unrecognized file extension" followed by the formats).
- Writing goes through the same class: `mesh.write("plate.e")` converts a `.mat` mesh to ExodusII, which VisIt can open on its own (chapter 28).
- The 2014 deck's "There are many other options (look at the libmesh MeshInput link)" is this table.

THE REST OF `main()`: WHAT IS NEW FOR IBFE, WHAT IS CHAPTER 5'S

line	what it does
`new IBFEMethod("IBFEMethod", db, &mesh, max_levels, true, restart_read_dirname, restart_restore_num)`	the IB operations for the mesh; `register_for_restart = true` so that the mesh data goes into the restart files
`new IBExplicitHierarchyIntegrator(..., ib_method_ops, navier_stokes_integrator)`	the same time integrator as the IB examples; it does not care which `IBStrategy` it drives
`initializeFEEquationSystems();`	create the FE systems (position, velocity, force) on the mesh; the `EquationSystems` pointer is what the ExodusII writer writes
`std::unique_ptr<ExodusII_IO> exodus_io = uses_exodus ? std::make_unique<ExodusII_IO>(mesh) : nullptr;`	the plate's output file writer (libMesh); the course used `AutoPtr`, removed from libMesh in 2017
`exodus_io->append(from_restart);`	a restarted run adds its steps to the existing `output.ex2` instead of starting it over

THE REST OF `MAIN()` (2): OUTPUT, RESTART, POST-PROCESSING

line	what it does
`initializeFEData();` `initializePatchHierarchy(...)`	FE data first, then the fluid grid

`exodus_io->write_timestep(exodus_filename,`
`*equation_systems,`
`iteration_num / viz_dump_interval + 1,`
`loop_time);`
	one time state per visualization dump, numbered from 1

`writeFEDataToRestartFile(` `restart_dump_dirname, iteration_num)`	next to the SAMRAI restart file: both are needed to restart

`output_data(...)`:
`mesh.write(fe_mesh.NNNNN.xda),`
`equation_systems->write(...)`
	the post-processing dump (`data_dump_interval > 0`): the mesh and all FE data in libMesh's own format

The fluid solver choice, the initial and boundary condition objects, the `VisItDataWriter`, the time loop and the restart and timer dumps are the chapter 5 program line for line.

BUILD AND RUN

```
module load gnu13 openmpi5                          # Puma only; a laptop with autoibamr skips this
  line
source ~/autoibamr/configuration/enable.sh
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j 4
mpirun -np 4 ./build/main2d input2d                 # or: sbatch job.slurm
```

- `CMakeLists.txt` is the six-line file of chapter 3 (`find_package(IBAMR 0.16.0 REQUIRED)`, `target_link_libraries(main2d PRIVATE IBAMR::IBAMR2d)`); libMesh comes with IBAMR's targets, nothing to add. The 2014 Makefile needed `LIBSNEW = $(LIBS) -lnetcdf -lcurl` by hand (slide 48).
- 4000 steps of `DT = 1e-3` to $t = 4$; 41 visualization dumps; 26 s on four laptop cores with 0.16.0, about the same with 0.19.0. The two versions give the same positions to three digits.
- A misspelled `MOTION` stops the run at once:

```
Unknown MOTION = "ROTATE2" in the input file.
Valid options are: FULL, HALF, ROTATE
```

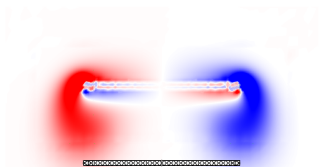
- Two course-era traps, fixed in the shipped files: the half-plate and rotate input files ended with a stray `\\` (an input parse error at start-up), and a file saved with Windows line endings stops with `Illegal character token in input` (chapter 6). The course hid the second by ending every line with `//`.

THE OUTPUT: TWO KINDS OF FILES

`viz_IB2d/dumps.visit`	the fluid on the adaptive grid, as in Part III: `U`, `P`, `Omega`, `F` (the spread force), `Div_U`, one `visit_dump.NNNNN` folder per dump
`viz_IB2d/output.ex2`	the plate, one ExodusII file with all 41 time states: the mesh (`Mesh`, one element block, node numbers) and the nodal variables `dX_0`, `dX_1` (displacement from the reference position), `X_0`, `X_1` (current position), `F_0`, `F_1` (force) and `U_0`, `U_1` (velocity)
`IB.log`, `IB2d.log`	the log (`AppInitializer`'s name in the program; `log_file_name` in `Main`)
`restart_IB2d/`	only with `restart_dump_interval > 0`: SAMRAI's files and `writeFEDataToRestartFile`'s
`hier_data_IB2d/`	only with `data_dump_interval > 0`: `hier_data.NNNNN.samrai.PPPPP` (HDF5), `fe_mesh.NNNNN.xda`, `fe_equation_systems.NNNNN`

- "`dumps.visit`: Eulerian data; `output.ex2`: finite element / Lagrangian data; Exodus II type file; can have multiple files of this type for different parts of object" (2014). With several meshes (chapter 21) IBAMR writes one `.ex2` per part.
- There is no `lag_data.visit` here: the plate is not a point set. VisIt reads ExodusII directly (next slides), and so do Python (`netCDF4`, `exodus.py`) and MATLAB (`ncread`); chapter 28 reads the positions of the tips from it.
- The 2020 deck wrote "`dump.visit`"; the file is `dumps.visit`.

SEEING THE PLATE IN VISIT: THE MESH IS DRAWN WHERE IT STARTED

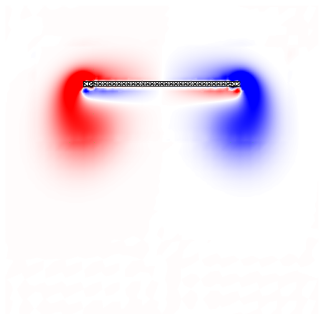


1. Open `viz_IB2d/dumps.visit`; **Add** → **Pseudocolor** → **Omega**; **Draw**.
2. Open `viz_IB2d/output.ex2` as a second source; **Add** → **Mesh** → **Mesh**; **Draw**.
3. Untick **Apply operators to all plots** (bottom left of the plot list), so that the next step touches only the mesh plot.

At $t = 1$ the vorticity shows the plate has moved up by 0.5, but the mesh is drawn at its *starting* position: ExodusII stores the reference mesh and the displacement `dX` separately. The next slide moves it.

> The 2014 README and every deck since teach this with a YouTube video by Shannon Jones (`watch?v=Kj_MACemSd0`); the IBFE Read Me adds “the youtube tutorial has a mistake, but this is corrected in the google slides.” The video is listed in chapter 0.1 and not reproduced.

SEEING THE PLATE IN VISIT (2): THE DX EXPRESSION AND THE DISPLACE OPERATOR

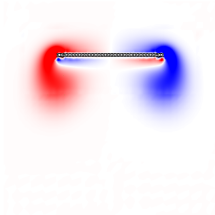


4. **Controls** → **Expressions** → **New**: name `dX`, type *Vector Mesh Variable*, definition `{dX_0, dX_1}`; **Apply**.
5. Select the Mesh plot; **Operators** → **Transforms** → **Displace**; in its attributes set *Displacement variable* to **Vectors** → **dX**; **Apply**; **Draw**.
6. Save the expression (**Expressions** → **Save** writes an XML file) or the whole session; you will do this for every IBFE run.

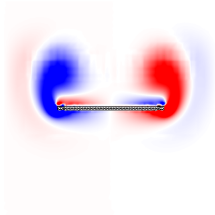
SEEING THE PLATE IN VISIT (3): OLDER OUTPUT, 3D, AND WHAT CHAPTER 28 ADDS

- Older IBAMR versions wrote the 2D mesh with three coordinates: then the definition was `{dX_0, dX_1, 0}` and the mesh had to be flattened with **Operators** → **Transforms** → **Project** (“the default projection should be for the *xy*-plane”). Current output is two-dimensional (`coordx`, `coordy` only), so `{dX_0, dX_1}` and no Project, as the 2020 note “for new ibamr install in 2D” says.
- In 3D the expression is `{dX_0, dX_1, dX_2}`.
- Chapter 28 continues: colouring the mesh by `F` or `U`, picking nodes, the scripted version.
- The figures of this chapter were made the scripted way (`wsl/final/visit_ch17.sh`): `DefineVectorExpression("dX", "{dX_0, dX_1}")`, `AddPlot("Mesh", "Mesh")`, `AddOperator("Displace")` with `DisplaceAttributes().variable = "dX"`. Chapter 32 has the whole recipe.

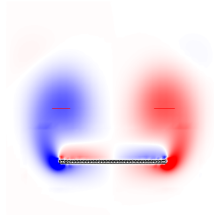
FULL: UP, DOWN, RELEASED



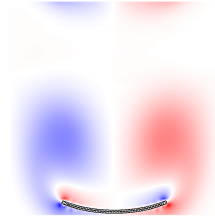
$t = 1$: top of the motion



$t = 2$: on the way down



$t = 3$: released at $y = -0.5$

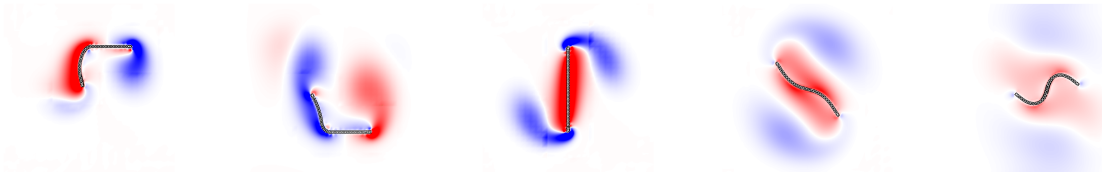


$t = 4$: still sinking, ends lagging

- Vorticity ± 5 (red positive, blue negative) on the fluid grid; the plate's mesh displaced by `dX`. The window is the whole 2×2 periodic box's centre $[-1, 1]^2$.
- A plate pushed broadside sheds a vortex pair from its ends; the pair follows it down. After the release at $t = 3$ the fluid around the plate is still moving down and carries the plate with it: $y = -0.729$ at $t = 3.5$ and -0.936 at $t = 4$, with the ends behind the middle (-0.877). Nothing stops it but viscosity; in a longer run it would leave the box through the periodic bottom and reappear at the top.

IBFE/01, `MOTION = "FULL"`, IBAMR 0.19.0; `wsl/final/visit_ch17.sh`.

HALF AND ROTATE



HALF, $t = 1$ HALF, $t = 3$ ROTATE, $t = 1$ ROTATE, $t = 2$ ROTATE, $t = 4$

- HALF: the right tip follows the target exactly ($y = +0.500$ at $t = 1$, -0.500 at $t = 3$); the free left half lags, so the plate tilts by about $+39^\circ$ at $t = 1$ and about -30° from $t = 2$ on, and bends at the join between tethered and free material. The material law alone carries the left half.
- ROTATE: a quarter turn by $t = 1$, as prescribed, with the centre at the origin; released, the swirl the turn made keeps turning the plate, to about 140° at $t = 2$ and 188° at $t = 4$, while it bends in the flow it created.

IBFE/01, `MOTION = "HALF"` and `"ROTATE"`, IBAMR 0.19.0.

WHAT WAS MEASURED

From `output.ex2` (the centre is the mean of the 101 nodes; the tips are the nodes that start at $x = \pm 0.5$ on the centre line); IBAMR 0.16.0 and 0.19.0 agree:

FULL, time	centre y	
0	0.000	
1.0	$+0.500$	top
2.0	0.000	
3.0	-0.500	released
3.5	-0.729	carried by the fluid
4.0	-0.936	ends at -0.877

run	result
HALF	right tip $y = +0.500$ at $t = 1$, -0.500 at $t = 3$; tilt $+39^\circ$ ($t = 1$), -30° ($t \geq 2$)
ROTATE	90° at $t = 1$; 140° at $t = 2$; 188° at $t = 4$; centre at the origin throughout

The 2014 and 2020 decks showed two renders each and no numbers; these are the acceptance test for the example: if your run gives these, your IBFE install works.

WHAT WAS MEASURED (2): A SOFTER TETHER, A SOFTER AND A STIFFER PLATE

change	result
`KAPPA_S = 1e4`	the plate trails its target ($y = 0.241$ instead of 0.250 at $t = 0.5$) and overshoots when it turns (0.507 at $t = 1$)
`SHEAR_MODULUS = 100`, HALF	fine; tilts less, about 32° at $t = 1$
`SHEAR_MODULUS = 1`, HALF	the free half crumples (tip from $x = -0.5$ to -0.15 by $t = 1.5$), the flow it stirs up is fast, and the run stops near $t = 1.6$ with `Time step size change encountered`; halving `DT` delays the stop to $t \approx 2.05$ (chapter 16)

These are the “Things to try” of the example’s README, run; chapter 8a’s rule applies here too: change one parameter, predict, run, compare.

HOW THE `.MAT` MESH WAS MADE: THE PDE TOOLBOX RECIPE

From the 2016 README (Julian Wag; the 2014 generic README promised this and never gave it). In MATLAB:

```
pdepoly([.5 .5 -.5 -.5], [.0156 -.0156 -.0156 .0156], 'g');   % the plate's outline, corners in
  order
```

This opens the PDE Modeler with the rectangle. **Mesh** → **Parameters**: maximum edge size two to four times the fine grid spacing (0.0312 was used); **Mesh** → **Initialize Mesh**; **Mesh** → **Export Mesh** with the default names `p`, `e`, `t`. Then write the file libMesh reads:

```
fid = fopen('IBFE_Mesh2D_128.mat', 'w');
fprintf(fid, '%d %d ', length(p), length(t));     % number of nodes, number of triangles
fprintf(fid, '%f %f ', p);                        % x y of each node
fprintf(fid, '%d %d %d %d ', t);                  % three node numbers and the subdomain of each
  triangle
fclose(fid);
```

- `p` is $2 \times N$ (coordinates), `t` is $4 \times M$ (three nodes and a subdomain number); `fprintf` writes them column by column, which is the order of slide 10.
- The README printed `fopen(IBFE_Mesh2D_128.mat'` with a curly quote and a missing one, which MATLAB rejects; the line above is the corrected one.
- The `.mat` name is libMesh's convention for this text format; do not `save` a MATLAB binary file under it.
- Without MATLAB: make the same plate with gmsh (chapter 18: a rectangle, `Mesh.CharacteristicLengthMax = 0.03`, export MSH 2.2 ASCII) and set `MESH_FILENAME = "plate.msh"`; or with Triangle from a `.poly` file (next slide).

THE .POLY FILE THAT DESCRIBES A DIFFERENT PLATE

Every course copy of this example shipped `Beam_IBFE_Mesh2D.m`. It does not write the `.mat` mesh; it writes a Triangle input file, `IBFE_Mesh2D_128.poly`, for a rectangle 1.0×0.25:

```
L = 1.0; W = 0.25;                % the beam's length and width
...
fid = fopen('IBFE_Mesh2D_128.poly','w');
fprintf(fid,'%d 2 0 0\n', Npts);  % nodes, dimension 2, no attributes, no boundary markers
...                               % the node lines, then the segment lines
```

- The plate the example runs is 1.0×0.0312, eight times thinner, with 101 nodes; the `.poly` rectangle, meshed with Triangle (`triangle -pq IBFE_Mesh2D_128.poly` gives `.node` and `.ele` files that libMesh reads, slide 33), is a different structure with different dynamics.
- The crosswalk found the mismatch when the example was ported; the script and its output are kept in `extras/` with a note, because a `.poly` file is one honest way to make a triangle mesh (chapter 18), and because readers of the old folders will meet it.
- `extras/beam_ibfe_mesh2d.py` writes an identical `.poly` without MATLAB.

> When an example ships a generator script, check that the file it writes is the file the program reads: the name, the size and the number of nodes. Here two of the three differ.

WHAT CAN GO WRONG

symptom	cause and fix
`Unknown MOTION = ...`	the key is misspelled; the valid words are printed with the error
`Illegal character token in input` at start-up	Windows line endings, or the stray `\\` at the end of the course input files; save as LF, delete the backslashes
`ERROR: Unrecognized file extension` and a list of formats	the mesh file's extension is not one libMesh knows (slide 33); rename, or convert with gmsh
no `output.ex2`	`viz_writer` lacks `"ExodusII"`, so `uses_exodus` is false
the mesh does not move in VisIt	no Displace operator, or its variable is not `dX`; "Apply operators to all plots" was on and the operator landed on the fluid plot too
"mesh is 3D" in VisIt, plate invisible against the fluid	an old IBAMR's three-coordinate 2D output: `{dX_0, dX_1, 0}` and the Project operator
`Time step size change encountered`	the plate is too soft for the flow it creates (`SHEAR_MODULUS = 1`): a stiffer plate, a smaller `DT`, or accept it (chapter 16)
fluid leaks through the plate	too few quadrature points: raise `IB_point_density`; or the mesh is much coarser than the grid
a run restarted from `restart_IB2d` starts `output.ex2` over	the writer was not told to append: `exodus_io->append(from_restart)` before the first write
`'AutoPtr' was not declared`, `libMeshEnums`	a course-era program against a current libMesh (slide 48)

WHAT THE OLDER DECKS HAD THAT IS NOW PART X

2014–2020	now
Makefile: `LIBSNEW = $(LIBS) -lnetcdf -lcurl`, `IBAMR_SRC_DIR = /nas02/apps/ibamr-3803/...` or `.../ibamr-2016-11/...`, a `main-2d:` rule with `$(LIBSNEW)` (the 2014 deck's slides 6–7; the 2020 deck's slide 6 with `/groups/lauram9/ib10/ibamr/...`)	`CMakeLists.txt` with `find_package(IBAMR)`; libMesh, netCDF and curl come with the `IBAMR::IBAMR2d` target (chapter 3; the old flags in chapter X2)

`LibMeshInit init(argc, argv);`
`SAMRAI_MPI::setCommunicator(PETSC_COMM_WORLD);`
`SAMRAIManager::startup();`
	`IBTKInit ibtk_init(argc, argv, MPI_COMM_WORLD);`

`Mesh mesh(NDIM)` (ibamr/3803)	`Mesh mesh(init.comm(), NDIM)`

WHAT THE OLDER DECKS HAD THAT IS NOW PART X (2)

2014–2020	now
`target_force_function(F, FF, X, s, elem, const vector<NumericVector<double>*>& system_data, time, ctx)`: seven arguments	the nine of slide 17: `var_data` and `grad_var_data` in place of `system_data`
`libMeshEnums::Order`	`libMesh::Order`
`AutoPtr<ExodusII_IO> exodus_io(new ExodusII_IO(mesh))`	`std::unique_ptr<ExodusII_IO>`
`#include <IBAMR_config.h>`, `<IBTK_config.h>`	removed
`s_dump(1)=s(1)+s(0)*sin(2*3.14*0.5*time)`	an exact rotation with `PI` (slide 23)

The old programs compile against the 0.16.0–0.19.0 headers only after all of these; chapter X3 lists the error messages each one produces (`'AutoPtr' was not declared`, `fatal error: IBAMR_config.h`, the `IBFEMethod(...)` constructor mismatch of the multi-mesh programs).

EXERCISES

1. Translate the plate to the left at 0.3 for one second (the 2014 slide 10), then release it. Where is its centre at $t = 2$? Compare with FULL's drift after release.
2. Add the sinusoidal body force of slide 25 (a new `MOTION = "SHAKE"` case, with the amplitude and frequency in the input) and run it for four periods, once on the ends as the 2016 code does and once on the middle half as the 2014 slide says. Which part of the plate moves most, and why is it not the forced part alone?
3. Repeat FULL with `KAPPA_S = 1e4` and `1e2`. At which stiffness does the plate stop following the target, and what does the vorticity look like then?
4. Repeat HALF with `SHEAR_MODULUS = 1` and watch the run stop. Read chapter 16's slide on `Time step size change encountered`, then find the largest `DT` that reaches $t = 4$.
5. Make the same plate in gmsh (chapter 18), run FULL with `MESH_FILENAME` set to it, and compare the centre positions of slide 43. Then halve the element size and compare again.
6. Put a pressure load of slide 30 on the plate (no tether: `MOTION = "FULL"` with `KAPPA_S = 0`) and explain what a uniform pressure on a closed boundary does to a nearly incompressible body.

RULES OF THUMB

- Four things: a mesh, a stress function, a force function, the registrations. Everything else is the chapter 5 program.
- Write prescribed motion as a target position in the reference configuration; release by setting the target to the current position.
- `s(0)` is x and `s(1)` is y in the reference configuration; `X` is the current position. IBAMR's headers call them `X` and `x`.
- One body force function per part, any number of stress functions; each stress function gets its own quadrature order.
- `viz_writer = "VisIt","ExodusII"`: both words, or there is no plate file.
- In VisIt the mesh is drawn where it started until you displace it by `dX = {dX_0, dX_1}`.
- `MFAC` and `ELEM_TYPE` do nothing to a mesh read from a file; `IB_point_density` does.
- Check that a shipped generator writes the file the program reads.

SUMMARY

1. IBFE replaces points, springs and beams by a finite element mesh with a material law. Forces come from a first Piola–Kirchhoff stress function and from body and surface force functions, all C++ callbacks with the nine- (or twelve-) argument signature, and are spread from quadrature points.
2. Example IBFE/01 is the course's three tether examples in one program: a plate pulled up and down, half of it, or turned; `MOTION` picks one. It builds with the six-line CMake file and runs in under a minute; the numbers of slide 43 are its acceptance test.
3. The input adds a handful of IBFE keys; `IB_point_density` and the quadrature orders matter for any mesh, `MFAC` and `ELEM_TYPE` only for a mesh built in the program.
4. The plate's output is one ExodusII file with the reference mesh and `dX`; VisIt needs the `dX` expression and the Displace operator to draw it where it is.
5. Ten years of versions left a handful of slips: the `s(1)` comment, the sheared "rotation", $2 \cdot 3.14$, the half-turn that was called a spin, the `.poly` generator for another plate, the curly quote in the README, the "1.0 is better" comments. All are corrected in the example and recorded here.

Chapter 18 makes meshes: gmsh, Triangle and TetGen, Coreform Cubit, libMesh's generators, and the element-size rules. Chapter 19 writes the stress functions properly.

RECORDINGS (LISTED, NOT INCORPORATED)

Shannon Jones's YouTube tutorial on visualizing IBFE boundaries in VisIt (`youtube.com/watch?v=Kj_MACemSd0`; the IBFE Read Me warns that it has a mistake the slides correct), and the 2020 Zoom recordings of the IBFE lectures, are listed in the front matter's video list (chapter 0.1). Nothing in them is reproduced here.

REFERENCES AND CREDITS

- B. E. Griffith and X. Luo, *Hybrid finite difference/finite element immersed boundary method*, International Journal for Numerical Methods in Biomedical Engineering 33 (2017) e2888. https://doi.org/10.1002/cnm.2888; https://arxiv.org/abs/1612.05916.
- IBAMR 0.19.0: `include/ibamr/IBFEMethod.h`, `include/ibamr/FEMechanicsBase.h`, `src/IB/IBFEMethod.cpp` (`getFromInput`), `ibtk/include/ibtk/libmesh_utilities.h`, `ibtk/include/ibtk/IBTKInit.h`; https://github.com/IBAMR/IBAMR. libMesh 1.7.8: `src/mesh/namebased_io.C`, `matlab_io.C`; https://libmesh.github.io.
- The example of this series: `examples/IBFE/01-tethered-plate` (`main.cpp`, `input2d`, `IBFE_Mesh2D_128.mat`, `CMakeLists.txt`, `job.slurm`, `README.md`, `extras/`); figures by `wsl/final/visit_ch17.sh`.
- Sources: Alex Hoover, *1-IBFE Tutorial: Getting Started* (UNC, 2014, edited 2017, 18 slides) and the examples' README files (Hoover 2014; Julian Wag 2016); *1-IBFE Getting Starting and Visualizing a Mesh* (MATH 769, 2018, 27 slides; lab versions 2020–2024, 26 slides); *IBFE Read Me* (2017). The course examples were developed with Alex Hoover, Julian Wag and Nick Battista (2014–2018) and revised at the University of Arizona (2022–2024).